为了确保责任界限清晰，最大化开发效率，我们将 1 个 Master Agent + 5 个 Slave Agents 以及 前端可视化系统 按照 “技术栈相似性” 和 “数据流向强关联性” 进行归类。

以下是 4 名成员 的具体模块认领清单。原则是：每个 Agent 只有唯一的“责任人”，但一个人可以负责多个 Agent。

成员 A： **负责模块： Master Agent (主控) + Slave Agent 4 (运筹优化)**

1. Master Agent（调度决策与优化中枢）

任务 1（连活算法实现）： 按照 PDF ，编写核心函数 identify_continuous_activity()。

输入： **任务列表。**

逻辑： 筛选所有“结束时间”与下一任务“开始时间”间隔 < 2 小时，且距离 < X 海里的任务对。

输出： 标记出哪两单任务是“连活单”。

任务 2（决策融合）： 编写主控逻辑，接收 Agent 2 的“违规名单”和 Agent 3 的“疲劳名单”，从候选拖轮中剔除这些船，然后调用 Agent 4 计算最优解

2. Slave Agent 4（运筹规划智能体）

任务 1（多目标评分）： 实现 PDF 的三个目标函数。

calc_cost(): 计算燃油成本（距离 x 油价）

calc_balance(): 计算方差（作业均衡度）。

calc_efficiency(): 计算等待时间。

任务 2（方案生成）： 不需要手写复杂的 NSGA-II 完整进化过程，Demo 版建议写一个加权评分脚本。

生成 Top-3 方案：一个侧重省油（权重 0.8），一个侧重均衡（权重 0.8），一个综合分最高。

成员 B：大模型与规则风控

负责模块： Slave Agent 2 (规则) + Slave Agent 5 (分析学习)

3. Slave Agent 2（规则与知识守护智能体）

任务 1（向量数据库搭建）： 将 PDF 中的 42 条规范（如“老旧拖轮回避高危作业”）转化为规则向量。任务 2（合规检查器）： 编写 check_rule(tug_id, job_id) 接口。

逻辑： 拿任务属性去库里搜。如果搜到“名称相似互斥”且命中，返回 False。

输出： 具体的违规原因文本（用于前端弹窗）。

4. Slave Agent 5（分析与学习智能体）

任务 1（自然语言解释）： 接入 DeepSeek/GPT API。接收成员 A 算出的方案数据，生成一段话：“本次调度优先选择了 75 区拖轮，因为处于连活窗口内。”

任务 2（反事实推演脚本）： 预设几个“剧本”。

当用户问“为啥不派亚洲 2 号？”时，Agent 5 自动调用 Agent 2 的检查结果，通过 LLM 生成回答：“经推演，如果派遣亚洲 2 号，将触发‘名称混淆’风险，且该船今日作业已饱和。

成员 C：数据仿真与状态管理 (The World)

负责模块： Slave Agent 1 (感知) + Slave Agent 3 (疲劳)

5. Slave Agent 1（全域感知智能体）

任务 1（泊位堆栈逻辑）： 这是 PDF 的核心痛点之一。

编写一个 Python Class BerthStack。模拟泊位只能“后进先出”。

演示逻辑： 如果系统想用“内档”的船，必须先生成一个“移泊”任务把“外档”船挪走。

任务 2（隐性任务生成）： 随机生成一些“辅助带缆”或“航道清理”的隐性任务，占用拖轮时间，看系统能否应对。

6. Slave Agent 3（疲劳度管理智能体）

任务 1（疲劳模型 BFM）： 实现累加器。

公式：当前疲劳 = 初始值 + (工作时长 * 1.0) + (夜间工作时长 * 1.5)。

任务 2（状态锁）： 维护 15 艘船的实时状态表。当某艘船疲劳值 >10（红区）时，将该船状态字段标记为 LOCKED_BY_FRMS，供成员 A 读取。

成员 D：前端交互与可视化 (The Face)

负责模块： Web 系统前端 + GIS 地图交互

虽然这不是一个“Agent”，但它是整个系统的载体。没有成员 D，其他三个人的代码只是控制台的 Log。

7. 综合演示系统 (Frontend)

任务 1（GIS 地图大屏）：

使用 Leaflet/Mapbox，在青岛港地图上画出泊位和航道。

对接成员 C 的数据，实时显示拖轮位置。

关键开发： 点击泊位，弹窗显示“内/外档”堆栈结构（可视化 Agent 1 的成果）。

任务 2（调度交互区）：

左侧：待排任务列表。

中间：成员 A 算出的 Top-3 方案卡片。

关键开发： 点击“一键采纳”按钮，动画演示拖轮连线（可视化连活逻辑）。

任务 3（Chatbot 窗口）：

右下角悬浮窗。对接成员 B 的 Agent 5 接口。

显示成员 B 生成的解释文字和反事实推演结果。

第一阶段：基础设施与数据底座（The Foundation）

目标： 让系统跑起来，地图有东西显示，后台有数据结构。

关键依赖： A 和 C 必须先商定数据格式（JSON）。

成员	任务详情	交付物
C	1. **数据构造：** 伪造 15 艘拖轮的静态数据（ID、马力、初始位置），伪造青岛港 35 个泊位的坐标数据。 2. **状态定义：** 定义好“疲劳值”和“泊位堆栈”的 Python 类结构。	tugs.json berths.json BerthStack.py
A	1. **接口定义：** 搭建 FastAPI 框架，定义 POST /schedule 接口的输入输出格式。 2. **算法框架：** 写好 Master Agent 的空函数壳，准备接入 C 的数据。	API 文档 (Swagger) scheduler.py （框架）
D	1. **地图初始化：** 搭建 Web 项目，加载 Leaflet/Mapbox 地图，把 C 提供的泊位坐标在地图上打点。 2. **静态展示：** 在地图上把 15 艘拖轮画出来（不动也行）。	前端原型页面 地图点位展示
B	1. **规则提取：** 从 PDF 中人肉提取 42 条规则，整理成 .txt 文档。 2. **知识库环境：** 本地部署一个轻量级向量库（如 ChromaDB）。	rules.txt 向量库环境

第二阶段：核心逻辑与单体智能（The Core Logic）

目标： 四个 Agent 各自的功能跑通，能算出结果，能拦截违规。

成员	任务详情	交付物
C	**1. 疲劳计算实现 (Agent 3)：** 写好公式，输入时间，输出疲劳值。 **2. 泊位逻辑实现 (Agent 1)：** 实现“外档不走，内档出不来”的逻辑判断。	FatigueModel.py check_berth_availability()
A	**1. 连活算法 (Master)：** **(核心)** 编写函数识别“时间 < 2h”且“距离近”的任务对。 **2. 评分逻辑 (Agent 4)：** 写一个简单的加权公式，给不同拖轮打分。	identify_chain_jobs() rank_solutions()
D	**1. 状态可视化：** 根据 C 的逻辑，给地图上的拖轮加颜色（红=疲劳，绿=健康）。 **2. 任务列表：** 左侧边栏显示待排任务。	带有状态颜色的拖轮图标 任务列表 UI
B	**1. 规则检索 (Agent 2)：** 写好 Python 函数，输入“拖轮 ID+任务 ID”，返回“是否违规”。 **2. Prompt 设计：** 调试 LLM 的提示词，确保能生成像样的解释。	check_compliance() explain_prompt.md

🟦 第三阶段：系统集成与联调（Integration）

目标： 后端逻辑串联，前端能通过按钮触发后台计算并显示结果。

成员	任务详情	交付物
A	**全流程串联：** 1. 从 C 获取拖轮状态（剔除疲劳船）。 2. 调用 B 的接口（剔除违规船）。 3. 运行连活算法和评分。 4. 输出 Top-3 方案给前端 D。	完整的 /schedule 接口响应
D	**交互开发：** 1. 点击“智能调度”按钮 -> 调用 A 的接口。 2. 接收数据，在界面下方渲染 3 张方案卡片。 3. **高亮连活：** 如果 A 返回了连活数据，在地图上画出金色的虚线。	完整的调度交互流程
B	**解释生成接入 (Agent 5)：** 当 A 算出方案后，B 自动把方案数据喂给 LLM，生成一段文	解释生成 API 联调完成

	字，通过接口返给 D。	
C	**场景数据锁死：** 配合大家调试，调整模拟数据。比如：故意把某艘船的疲劳值调高，验证 A 是否真的剔除了它。	调整后的 Mock 数据集

🗓 第四阶段：演示打磨（The Demo Script）

成员	任务详情	交付物
全员	**剧本彩排：** 确定演示的点击顺序。例如：先看地图 -> 再点调度 -> 再点反问。	演示脚本 Word 版
C	**构造“完美冲突”：** 造一个必须用“连活”才能解的局。 造一个必须“移泊”才能出的内档船。	demo_scenario_final.json
B	**反事实预设：** 针对剧本中未被选中的那艘船，预先生成好完美的解释文本，防止现场 LLM 胡说八道。	固定的回答文本
D	**视觉增强：** 1. 违规弹窗加红色震动特效。 2. 连活识别出来时，加一个“💰 节省 3000 元”的浮动标签。	最终演示版前端
A	**兜底逻辑：** 如果算法现场算不出最优解，写死一个 if scenario == 'demo': return fixed_best_plan，确保演示万无一失。	